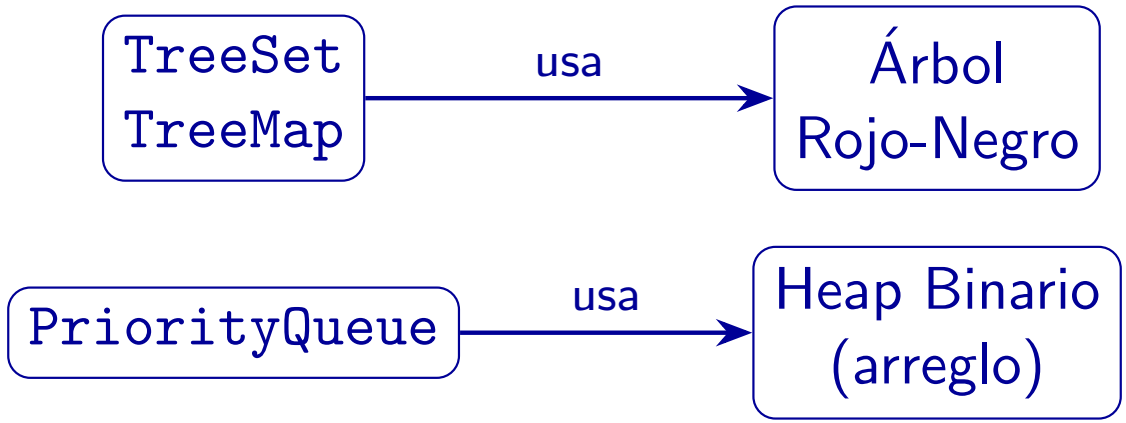
Ayudantía 21 Oct

TreeSet, TreeMap y PriorityQueue en Java

¿Qué son?

Son tres colecciones del paquete `java.util` que mantienen sus elementos **organizados según un orden** (natural con `Comparable` o externo con `Comparator`).

- `TreeSet<E>`: conjunto **ordenado** y **sin duplicados**.
- `TreeMap<K,V>`: mapa clave $\rightarrow$ valor **ordenado por clave**.
- `PriorityQueue<E>`: cola donde siempre sale primero el elemento de **mayor prioridad** (por defecto, el mínimo).



¿Para qué sirven?

TreeSet:

- Eliminar duplicados y mantener los datos permanentemente ordenados.
- Consultas de rango y vecinos: "el menor $\geq x$", "todos entre a y b".

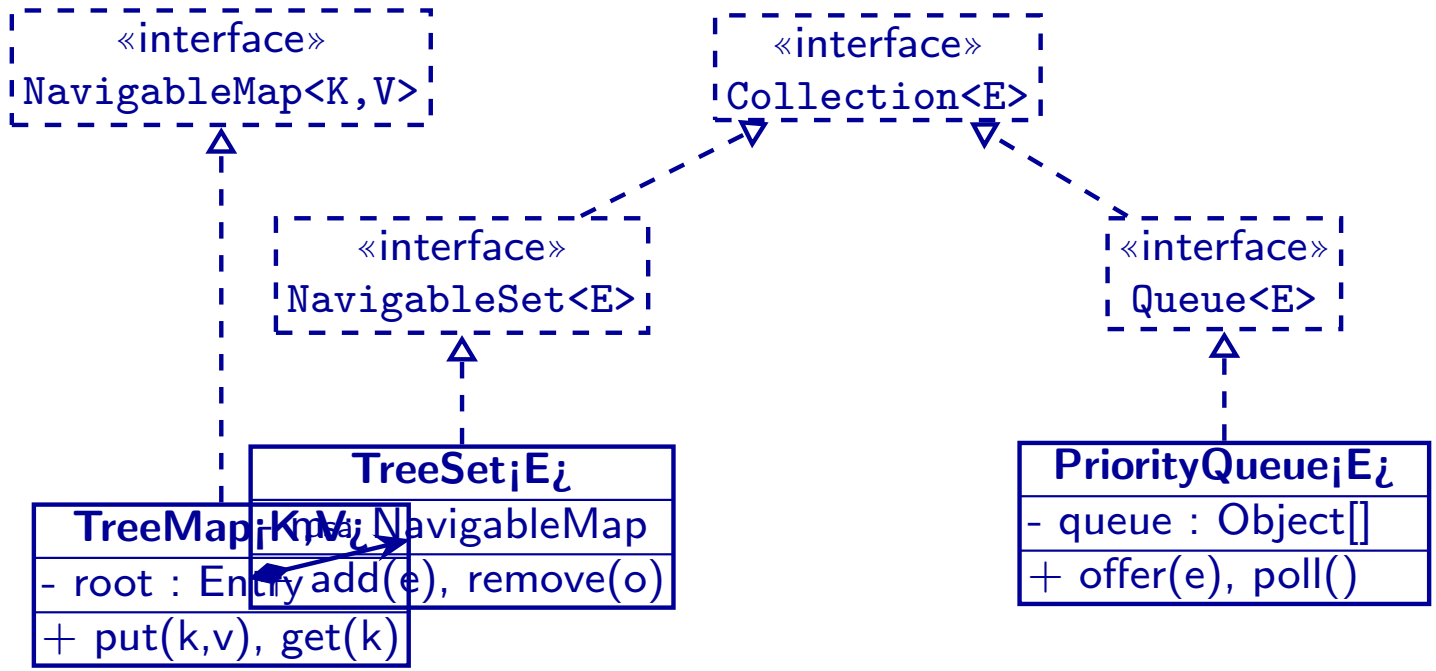
TreeMap:

- Diccionarios ordenados (ej: palabras en orden alfabético con sus conteos).
- Tablas por tramos (ej: nota $\rightarrow$ categoría mediante `floorKey`).

PriorityQueue:

- Atender eventos o tareas por urgencia (planificadores, simulaciones).
- Algoritmos clásicos: Dijkstra, Prim, Huffman, top-k mayores, Heapsort.

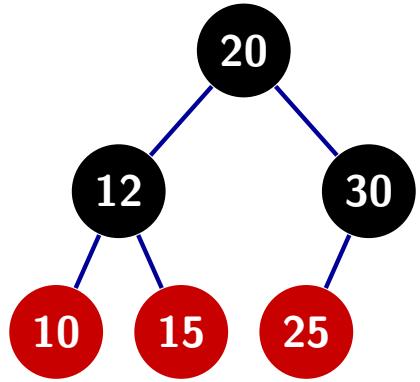
Jerarquía de clases (UML)



(`NavigableSet` extiende `SortedSet` → `Set`; `NavigableMap` extiende `SortedMap` → `Map`)

TreeMap y TreeSet: ¿Qué son?

- `TreeMap` es un **Árbol Binario de Búsqueda balanceado** del tipo **Rojo-Negro**.
- Cada nodo (`Entry`) guarda una **clave**, un **valor** y un **color**.
- `TreeSet` **no tiene árbol propio**: guarda sus elementos como **claves** de un `TreeMap` interno, con un valor ficticio (`PRESENT`).
- El balanceo garantiza altura $O(\log n) \Rightarrow$ todas las operaciones principales son $O(\log n)$.



Árbol Rojo-Negro: Propiedades

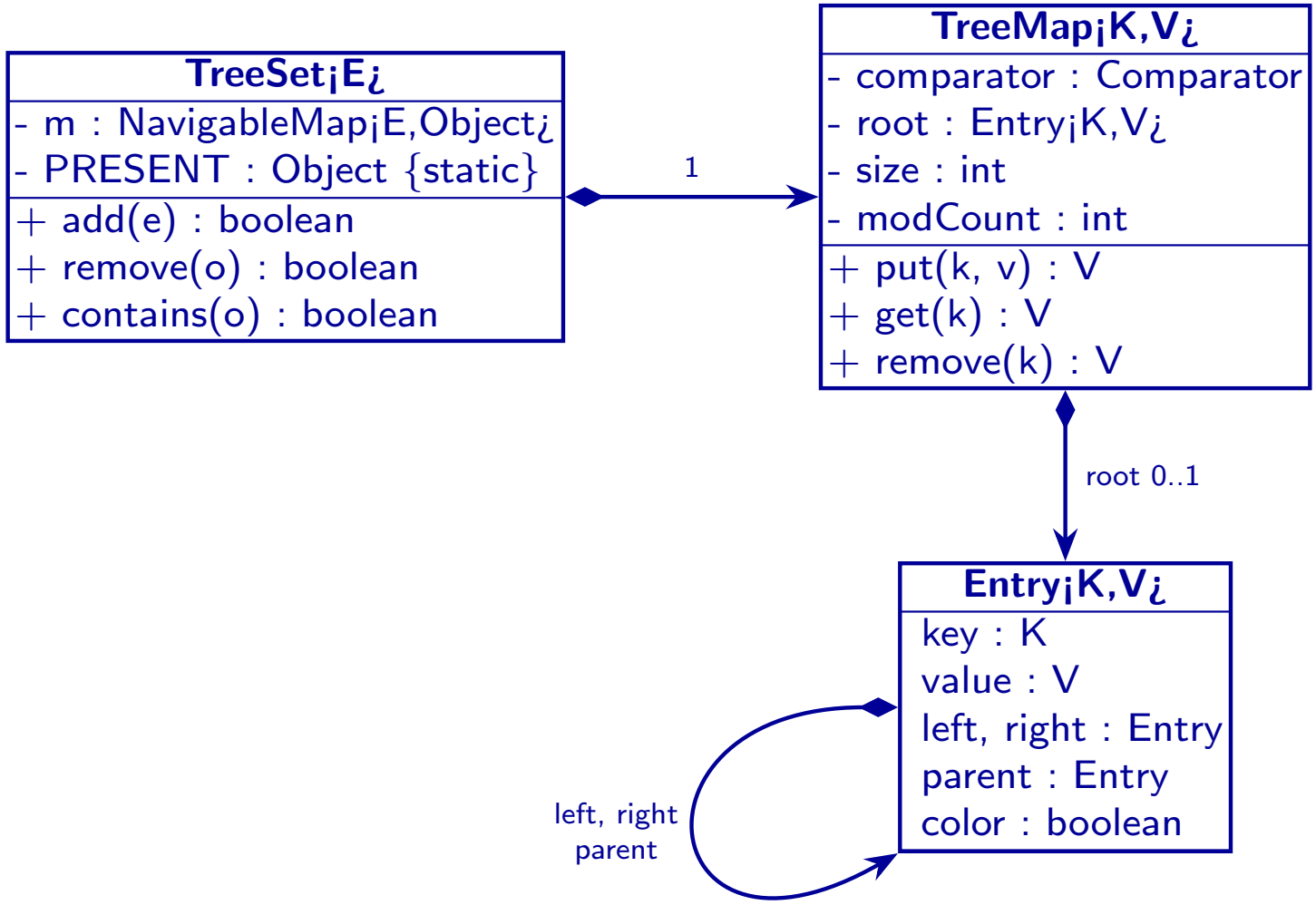
Es un BST donde cada nodo tiene un color y se cumplen **5 reglas**:

1. Todo nodo es **rojo** o **negro**.
2. La **raíz** es negra.
3. Las hojas nulas (`nil`) se consideran negras.
4. Un nodo rojo **no puede tener hijos rojos** (no hay dos rojos consecutivos).
5. Todo camino desde un nodo hasta sus hojas `nil` tiene la **misma cantidad de nodos negros** (altura negra constante).

Consecuencia matemática: el camino más largo es a lo más el doble del más corto:

$$h \leq 2 \log_2(n+1)$$

Estructura interna (UML)



`RED = false`, `BLACK = true`.

Estructura interna: TreeMap (Java)

```
public class TreeMap<K,V> extends AbstractMap<K,V>
        implements NavigableMap<K,V>, Cloneable, Serializable {

    private final Comparator<? super K> comparator;
    private transient Entry<K,V> root;   // raiz del arbol
    private transient int size = 0;
    private transient int modCount = 0;  // fail-fast

    private static final boolean RED   = false;
    private static final boolean BLACK = true;

    static final class Entry<K,V> implements Map.Entry<K,V> {
        K key;
        V value;
        Entry<K,V> left;
        Entry<K,V> right;
        Entry<K,V> parent;
        boolean color = BLACK;
        Entry(K key, V value, Entry<K,V> parent) {
            this.key = key; this.value = value; this.parent = parent;
        }
    }
}
```

Estructura interna: TreeSet (Java)

```
public class TreeSet<E> extends AbstractSet<E>
        implements NavigableSet<E>, Cloneable, Serializable
            {

    // El conjunto se guarda como las CLAVES de un TreeMap
    private transient NavigableMap<E,Object> m;

    // Valor ficticio asociado a cada clave
    private static final Object PRESENT = new Object();

    public TreeSet() {
        this.m = new TreeMap<>();
    }
    public boolean add(E e) {
        return m.put(e, PRESENT) == null;
    }
    public boolean remove(Object o) {
        return m.remove(o) == PRESENT;
    }
    public boolean contains(Object o) {
        return m.containsKey(o);
    }
}
```

Búsqueda: Definición

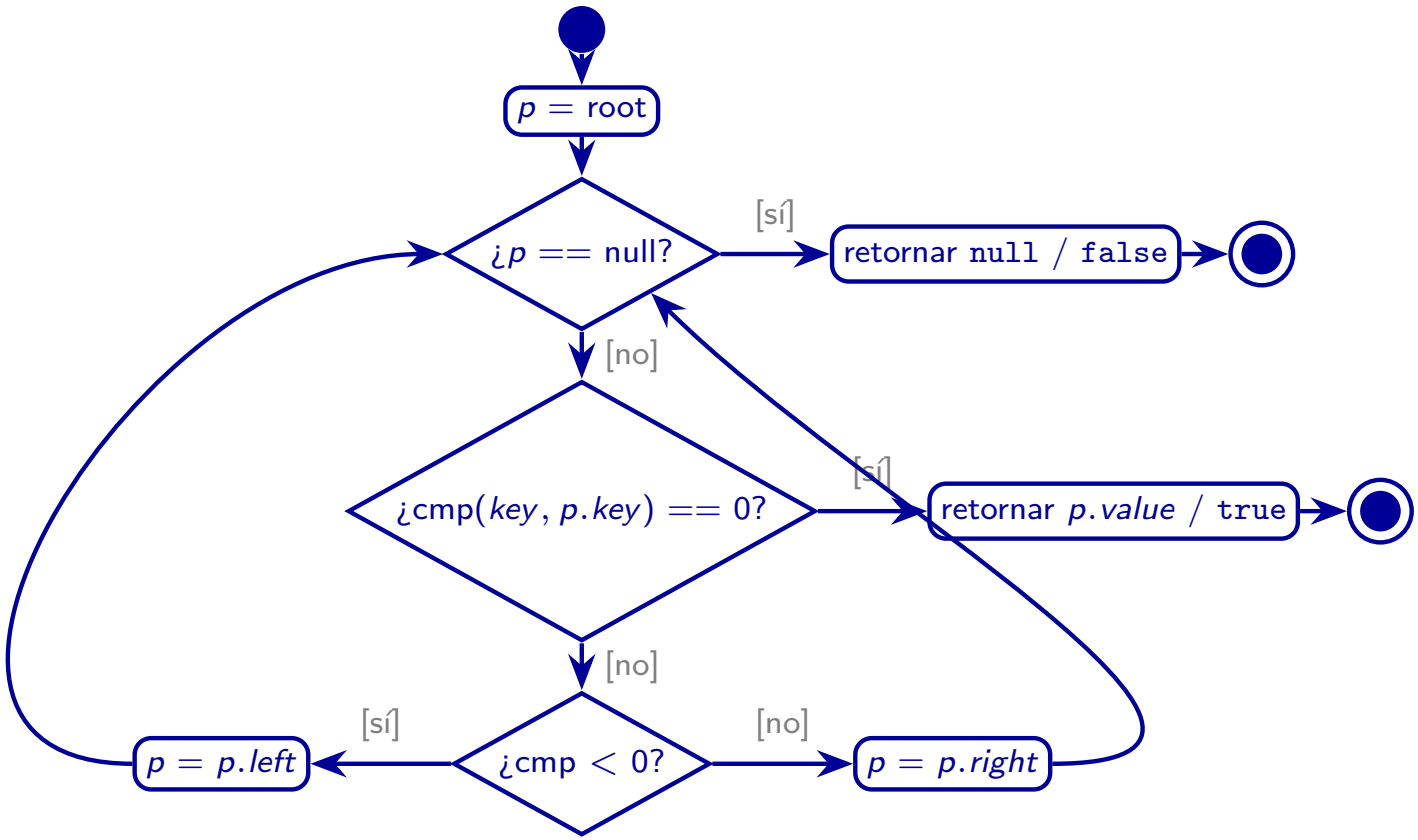
Igual que en un BST: se compara la clave buscada con la del nodo actual.

- Si es **menor** $\rightarrow$ se baja a la izquierda.
- Si es **mayor** $\rightarrow$ se baja a la derecha.
- Si es **igual** $\rightarrow$ encontrado.
- Si se llega a `nil` $\rightarrow$ la clave no existe.

Los colores **no se modifican ni consultan** durante la búsqueda. Como la altura es $O(\log n)$, el tiempo es $O(\log n)$.

- `TreeMap`: `get(k)`, `containsKey(k)`.
- `TreeSet`: `contains(o)`.

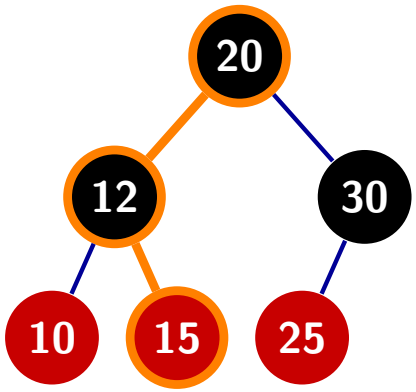
Búsqueda: Diagrama de Actividad UML



Cada iteración reduce la búsqueda a un subárbol $\Rightarrow$ a lo más $h \leq 2\log(n+1)$ pasos.

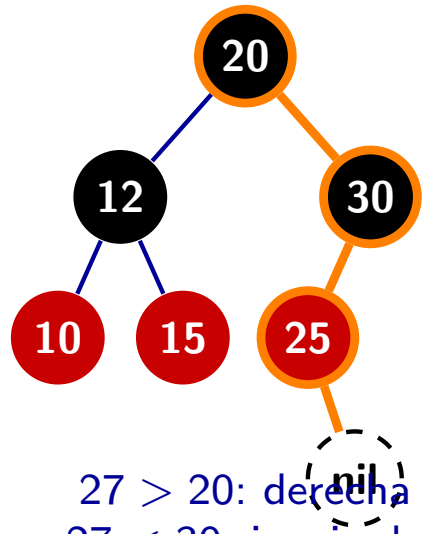
Búsqueda: Ejemplo paso a paso (Árbol)

buscar(15)



$15 < 20$: izquierda
$15 > 12$: derecha
$15 = 15$: **encontrado**

buscar(27)



$27 > 20$: derecha
$27 < 30$: izquierda
$27 > 25$: derecha $\rightarrow$ `nil`
no encontrado (`null`)

Búsqueda: Java (TreeMap.getEntry)

```
public V get(Object key) {
    Entry<K,V> p = getEntry(key);
    return (p == null ? null : p.value);
}

final Entry<K,V> getEntry(Object key) {
    if (comparator != null)
        return getEntryUsingComparator(key);
    if (key == null)
        throw new NullPointerException();
    Comparable<? super K> k = (Comparable<? super K>) key;
    Entry<K,V> p = root;
    while (p != null) {
        int cmp = k.compareTo(p.key);
        if (cmp < 0)        p = p.left;     // ir a la
            izquierda
        else if (cmp > 0)   p = p.right;    // ir a la derecha
        else                return p;       // encontrado
    }
    return null;                            // no existe
}
```

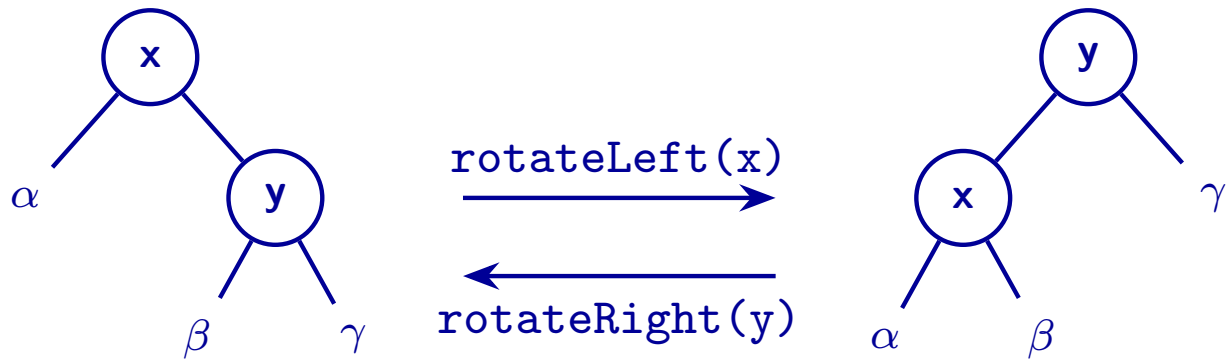
Inserción: Definición

1. Se desciende por el árbol **como en un BST** hasta encontrar la posición libre (`nil`).
2. Si la clave ya existe $\rightarrow$ se actualiza el valor (`TreeSet`: no se agrega).
3. El nuevo nodo se inserta como hoja y se pinta **ROJO** (así no altera la altura negra).
4. Si su padre también es rojo $\Rightarrow$ violación **rojo-rojo** $\Rightarrow$ **rebalancear** (`fixAfterInsertion`).
5. Al finalizar, la raíz siempre se pinta **negra**.

Costo total: $O(\log n)$ para descender + $O(\log n)$ recoloreos + **a lo más 2 rotaciones**.

Rebalanceo: Rotaciones

Una rotación reorganiza 2 nodos en $O(1)$ **manteniendo estrictamente el orden del BST**.



- El orden inorden se conserva intacto: $\alpha < x < \beta < y < \gamma$.
- El subárbol β cambia de padre: de y pasa a ser hijo de x.

Rebalanceo: Rotación en Java

```
private void rotateLeft(Entry<K,V> p) {
    if (p != null) {
        Entry<K,V> r = p.right;              // r sube
        p.right = r.left;                    // beta pasa a p
        if (r.left != null)
            r.left.parent = p;
        r.parent = p.parent;                 // r toma el lugar
            de p
        if (p.parent == null)
            root = r;
        else if (p.parent.left == p)
            p.parent.left = r;
        else
            p.parent.right = r;
        r.left = p;                          // p baja a la
            izquierda
        p.parent = r;
    }
}
// rotateRight(p) es estrictamente simetrico (left <->
    right)
```

Rebalanceo tras inserción: los 3 casos

Sea x el nodo rojo insertado, p su padre (rojo), a su abuelo y t su **tío**.

Caso 1: tío t es rojo $\rightarrow$ recolorear

- p y t pasan a negro, a pasa a rojo. Se repite el análisis desde $x \leftarrow a$.

Caso 2: tío t es negro y forma de "triángulo" (x es hijo derecho y p hijo izquierdo, o viceversa)

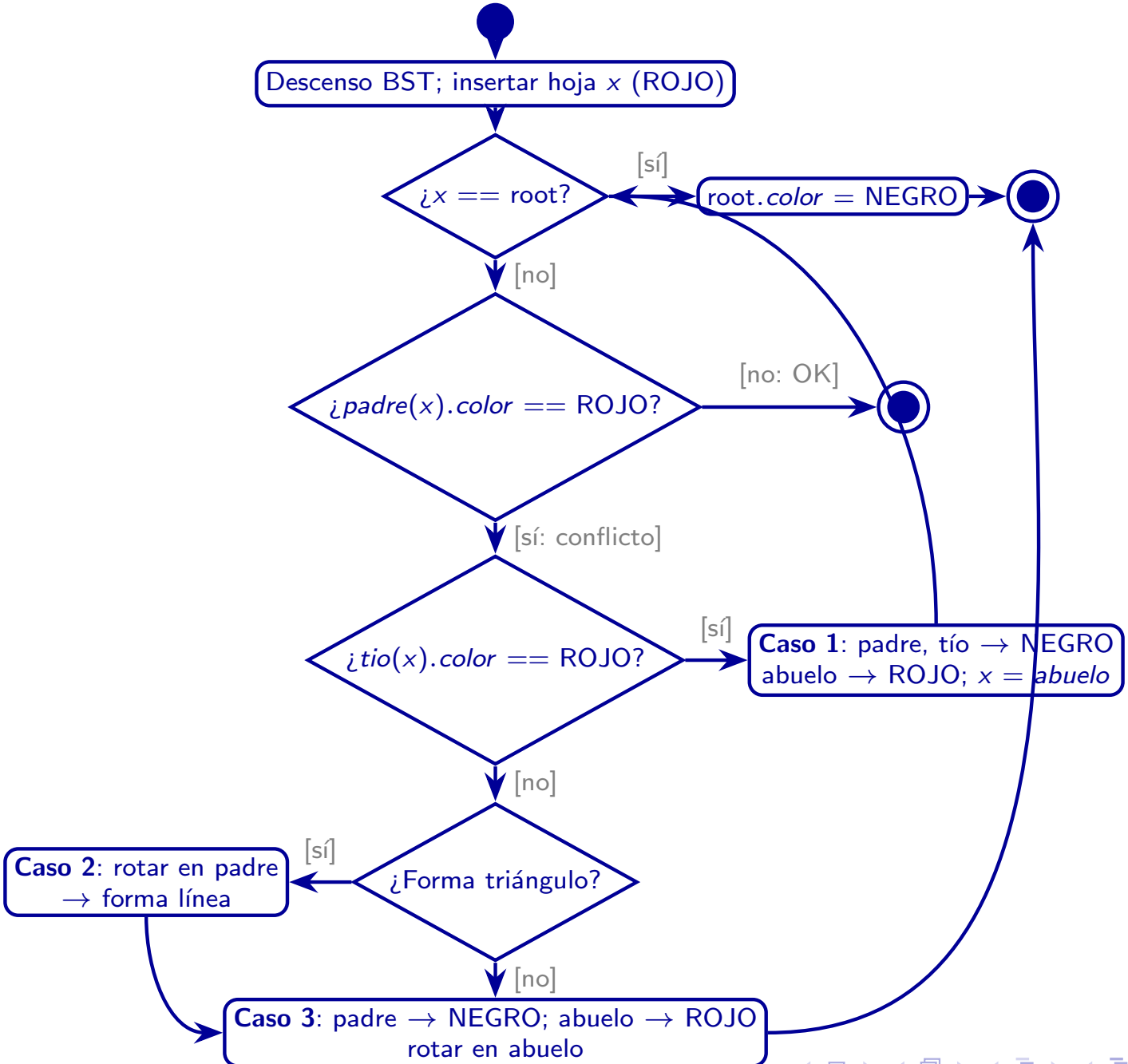
- Rotar en p para transformarlo en "línea" $\rightarrow$ pasa de inmediato al Caso 3.

Caso 3: tío t es negro y forma de "línea"

- p pasa a negro, a pasa a rojo y se **rota en** a hacia el lado contrario. **Termina.**

Nota: una referencia nula (`nil`) se considera un nodo **negro**.

Inserción y Rebalanceo: Diagrama de Actividad UML

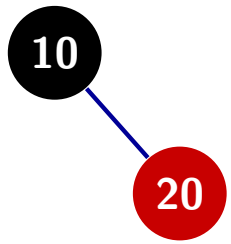


Inserción: Ejemplo paso a paso (1/4)

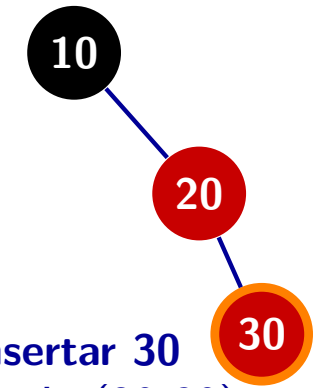
Insertar en secuencia: **10, 20, 30**, 15, 25, 12



insertar 10
árbol vacío:
raíz negra



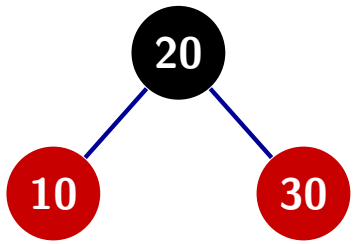
insertar 20
$20 > 10$, entra rojo.
padre negro: OK



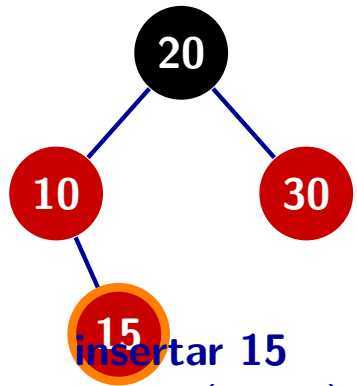
insertar 30
rojo-rojo (20-30)
tío `nil` = negro
forma de línea $\Rightarrow$ **Caso 3**

Inserción: Ejemplo paso a paso (2/4)

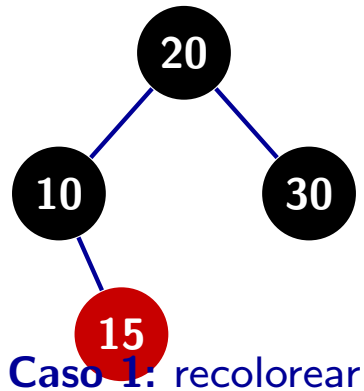
Insertar en secuencia: 10, 20, **30, 15**, 25, 12



Caso 3:
20 negro, 10 rojo
`rotateLeft(10)`



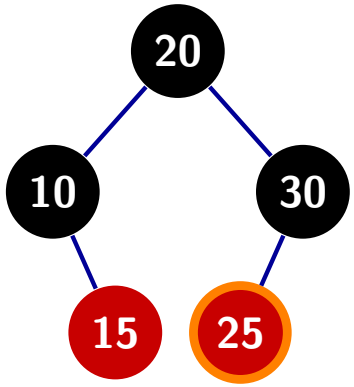
insertar 15
rojo-rojo (10-15)
tío 30 **rojo**
$\Rightarrow$ **Caso 1**



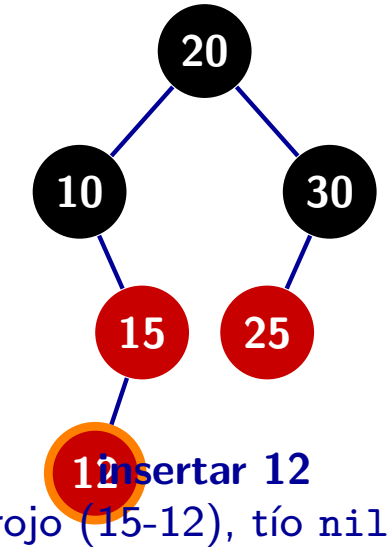
Caso 1: recolorear
10 y 30 negros, 20 rojo
20 es raíz $\Rightarrow$ negro

Inserción: Ejemplo paso a paso (3/4)

Insertar en secuencia: 10, 20, 30, 15, **25, 12**

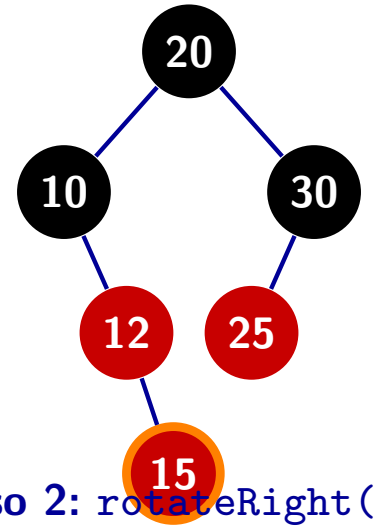


insertar 25
$25 > 20$, $25 < 30$
padre 30 negro: OK

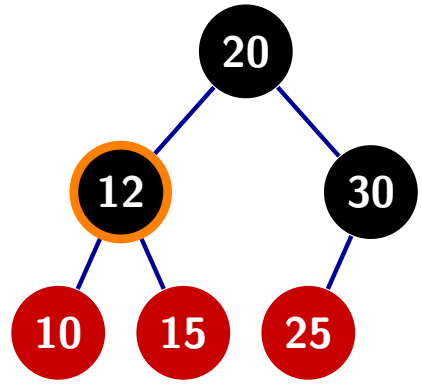


insertar 12
rojo-rojo (15-12), tío `nil` negro
12 izq. de 15, 15 der. de 10
forma de triángulo $\Rightarrow$ **Caso 2**

Inserción: Ejemplo paso a paso (4/4)



Caso 2: `rotateRight(15)`
ahora 10-12-15 es una línea
$\Rightarrow$ **Caso 3**



Caso 3: 12 negro, 10 rojo
`rotateLeft(10)`
¡Árbol válido!

Altura negra en todos los caminos $= 2$. Sin rojos consecutivos.

Inserción: Java (TreeMap.put)

```
public V put(K key, V value) {
    Entry<K,V> t = root;
    if (t == null) {                          // arbol vacio
        root = new Entry<>(key, value, null);  // raiz negra
        size = 1; modCount++;
        return null;
    }
    int cmp;
    Entry<K,V> parent;
    Comparable<? super K> k = (Comparable<? super K>) key;
    do {                                      // descenso tipo BST
        parent = t;
        cmp = k.compareTo(t.key);
        if (cmp < 0)       t = t.left;
        else if (cmp > 0)  t = t.right;
        else               return t.setValue(value); // ya existe
    } while (t != null);

    Entry<K,V> e = new Entry<>(key, value, parent);
    if (cmp < 0) parent.left = e;
    else         parent.right = e;
    fixAfterInsertion(e);                     // REBALANCEO
    size++; modCount++;
    return null;
}
```

Rebalanceo: Java (fixAfterInsertion 1/2)

```
private void fixAfterInsertion(Entry<K,V> x) {
    x.color = RED;                                  // entra rojo
    while (x != null && x != root && x.parent.color == RED) {
        if (parentOf(x) == leftOf(parentOf(parentOf(x)))) {
            Entry<K,V> y = rightOf(parentOf(parentOf(x)));  // tio
            if (colorOf(y) == RED) {                // Caso 1: tio rojo
                setColor(parentOf(x), BLACK);
                setColor(y, BLACK);
                setColor(parentOf(parentOf(x)), RED);
                x = parentOf(parentOf(x));
            } else {
                if (x == rightOf(parentOf(x))) {    // Caso 2: triangulo
                    x = parentOf(x);
                    rotateLeft(x);
                }
                setColor(parentOf(x), BLACK);       // Caso 3: linea
                setColor(parentOf(parentOf(x)), RED);
                rotateRight(parentOf(parentOf(x)));
            }
        } else {
            // continuacion rama simetrica en siguiente diapo...
```

Rebalanceo: Java (`fixAfterInsertion` 2/2)

```
        } else { // rama simetrica (padre es hijo derecho del abuelo)
            Entry<K,V> y = leftOf(parentOf(parentOf(x)));   // tio
            if (colorOf(y) == RED) {                        // Caso 1
                setColor(parentOf(x), BLACK);
                setColor(y, BLACK);
                setColor(parentOf(parentOf(x)), RED);
                x = parentOf(parentOf(x));
            } else {
                if (x == leftOf(parentOf(x))) {             // Caso 2
                    x = parentOf(x);
                    rotateRight(x);
                }
                setColor(parentOf(x), BLACK);               // Caso 3
                setColor(parentOf(parentOf(x)), RED);
                rotateLeft(parentOf(parentOf(x)));
            }
        }
    }
    root.color = BLACK;                                     // la raiz siempre negra
}
```

Eliminación: Definición

1. Se busca el nodo p a eliminar ($O(\log n)$).
2. **2 hijos:** se copia en p su **sucesor** (mínimo del subárbol derecho) y se procede a eliminar el sucesor (que tiene a lo más 1 hijo).
3. **1 hijo:** el hijo toma directamente el lugar de p.
4. **Hoja:** se desconecta del padre.

¿Cuándo se requiere rebalanceo?

- Si el nodo eliminado era **rojo** $\rightarrow$ no altera la altura negra; fin.
- Si era **negro** $\rightarrow$ ese camino pierde un negro: surge un **"doble negro"** y se invoca `fixAfterDeletion`.
- Si el nodo de reemplazo es rojo, basta con pintarlo negro.

Costo total: $O(\log n)$ con **a lo más 3 rotaciones**.

Rebalanceo tras eliminación: los 4 casos

Sea x el nodo "doble negro" y s su **hermano**.

Caso 1: hermano s es rojo

- s a negro, padre a rojo, rotar en el padre hacia x. Pasa a casos 2, 3 o 4.

Caso 2: hermano s es negro con ambos hijos negros

- s a rojo y el déficit de color negro **sube** al padre.

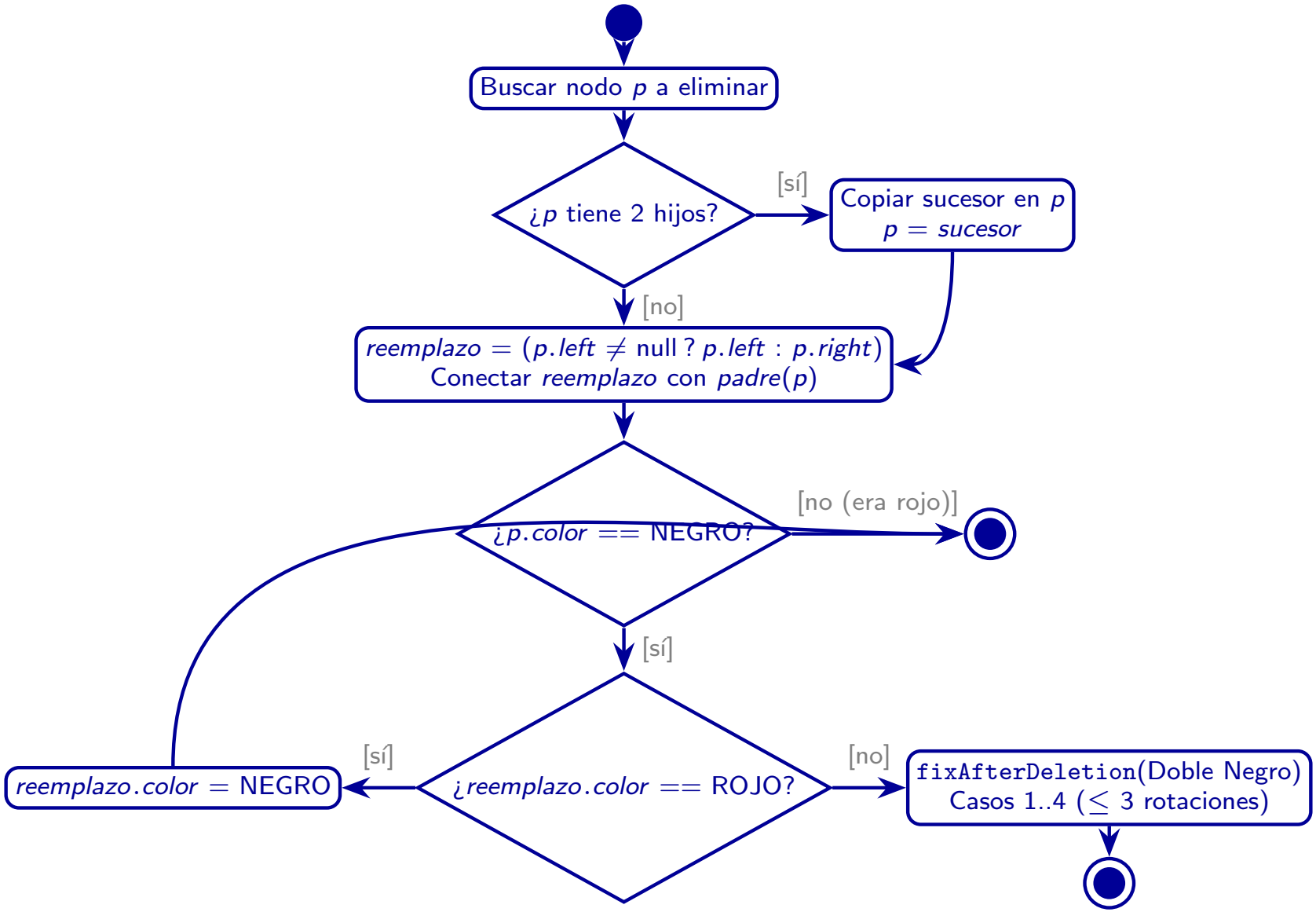
Caso 3: hermano negro, sobrino lejano negro y cercano rojo

- Sobrino cercano a negro, s a rojo, rotar en $s \rightarrow$ pasa al Caso 4.

Caso 4: hermano negro con sobrino lejano rojo

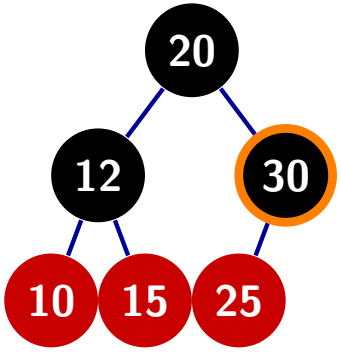
- s adopta el color del padre, padre y sobrino lejano a negro, rotar en el padre hacia x. **Termina.**

Eliminación y Rebalanceo: Diagrama de Actividad UML

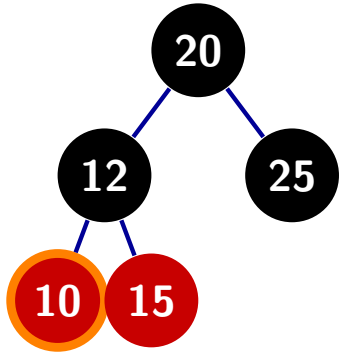


Eliminación: Ejemplo paso a paso (1/3)

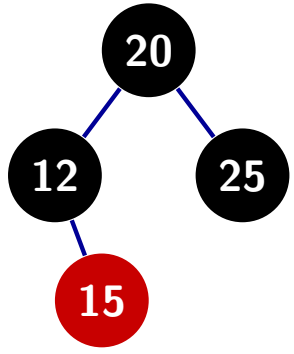
Eliminar en secuencia: **30, 10**, 25



eliminar 30
negro con 1 hijo (25)
el hijo toma su lugar



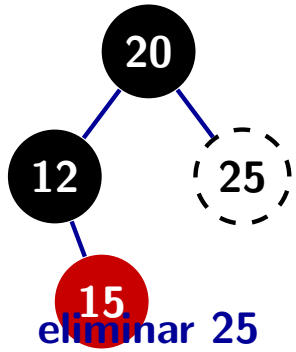
reemplazo 25 era rojo
$\Rightarrow$ se pinta **negro**
eliminar 10: hoja roja



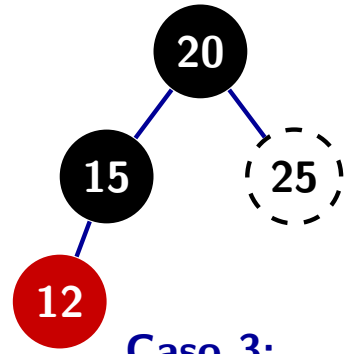
hoja **roja**:
se desconecta
sin rebalancear

Eliminación: Ejemplo paso a paso (2/3)

Eliminar en secuencia: 30, 10, **25**

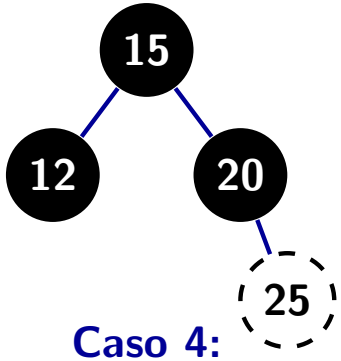


eliminar 25
hoja **negra** $\Rightarrow$ doble negro
hermano 12 negro
sobrino lejano (izq.) `nil`
sobrino cercano 15 rojo
$\Rightarrow$ **Caso 3**

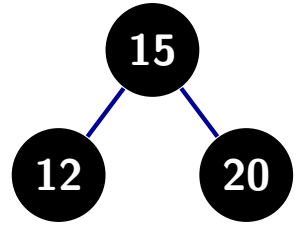


Caso 3:
15 negro, 12 rojo
`rotateLeft(12)`
ahora el sobrino lejano (12)
es rojo $\Rightarrow$ **Caso 4**

Eliminación: Ejemplo paso a paso (3/3)



Caso 4:
15 toma el color de 20 (negro)
20 y 12 a negro
`rotateRight(20)`



se desconecta 25
¡Árbol válido!
altura negra $= 2$

Eliminación: Java (TreeMap.deleteEntry)

```
private void deleteEntry(Entry<K,V> p) {
    modCount++; size--;
    if (p.left != null && p.right != null) {    // 2 hijos
        Entry<K,V> s = successor(p);
        p.key = s.key; p.value = s.value;       // copiar sucesor
        p = s;                                  // y borrar el sucesor
    }
    Entry<K,V> replacement = (p.left != null ? p.left : p.right);
    if (replacement != null) {                  // 1 hijo
        replacement.parent = p.parent;
        if (p.parent == null)           root = replacement;
        else if (p == p.parent.left)    p.parent.left  = replacement;
        else                            p.parent.right = replacement;
        p.left = p.right = p.parent = null;
        if (p.color == BLACK) fixAfterDeletion(replacement);
    } else if (p.parent == null) {              // unico nodo
        root = null;
    } else {                                    // hoja
        if (p.color == BLACK) fixAfterDeletion(p);
        if (p.parent != null) {
            if (p == p.parent.left)         p.parent.left = null;
            else if (p == p.parent.right)   p.parent.right = null;
            p.parent = null;
        }
    }
}
```

Rebalanceo: Java (fixAfterDeletion 1/2)

```
private void fixAfterDeletion(Entry<K,V> x) {
    while (x != root && colorOf(x) == BLACK) {
        if (x == leftOf(parentOf(x))) {
            Entry<K,V> sib = rightOf(parentOf(x));  //
                hermano
            if (colorOf(sib) == RED) {              // Caso
                1: hermano rojo
                setColor(sib, BLACK);
                setColor(parentOf(x), RED);
                rotateLeft(parentOf(x));
                sib = rightOf(parentOf(x));
            }
            if (colorOf(leftOf(sib))  == BLACK &&
                colorOf(rightOf(sib)) == BLACK) {   // Caso
                    2: sobrinos negros
                setColor(sib, RED);
                x = parentOf(x);                    //
                    deficit sube
            } else {
                // continuacion Casos 3 y 4 en siguiente
                    diapo...
```

Rebalanceo: Java (`fixAfterDeletion` 2/2)

```
                    if (colorOf(rightOf(sib)) == BLACK) { //
                        Caso 3
                        setColor(leftOf(sib), BLACK);
                        setColor(sib, RED);
                        rotateRight(sib);
                        sib = rightOf(parentOf(x));
                    }
                    // Caso 4: sobrino lejano rojo (termina)
                    setColor(sib, colorOf(parentOf(x)));
                    setColor(parentOf(x), BLACK);
                    setColor(rightOf(sib), BLACK);
                    rotateLeft(parentOf(x));
                    x = root;
                }
            } else { /* simetrico: left <-> right */ }
        }
        setColor(x, BLACK);
    }
```

Uso en Java: TreeSet y TreeMap

```
TreeSet<Integer> set = new TreeSet<>();
set.add(20); set.add(10); set.add(30); set.add(10);
System.out.println(set);                    // [10, 20, 30]
set.contains(20);                           // true
set.first();  set.last();                   // 10, 30
set.floor(25);  set.ceiling(25);            // 20, 30
set.headSet(20);                            // [10]
set.remove(10);                             // [20, 30]

TreeMap<String, Integer> map = new TreeMap<>();
map.put("rojo", 3); map.put("arbol", 2);
    map.put("heap", 1);
System.out.println(map);     // {arbol=2, heap=1,
    rojo=3}
map.get("heap");                            // 1
map.firstKey();                             // "arbol"
map.remove("rojo");

// Orden personalizado
TreeSet<Integer> inv = new
    TreeSet<>(Comparator.reverseOrder());
```

TreeSet y TreeMap: Propiedades principales

- **Ordenados**: la iteración recorre el árbol en **inorden** (de menor a mayor).
- **Sin duplicados**: `TreeSet` sin elementos repetidos; `TreeMap` sin claves repetidas.
- La igualdad se decide mediante `compareTo`/`compare` $= 0$ (**no** con `equals`).
- $O(\log n)$ garantizado en `add`, `remove`, `contains`, `get`, `put`.
- Métodos de navegación: `first`, `last`, `floor`, `ceiling`, `lower`, `higher`, `headSet`/`headMap`, `tailSet`/`tailMap`, `subSet`/`subMap`.
- **No admiten** `null` como elemento/clave con orden natural (`NullPointerException`). `TreeMap` sí admite valores `null`.
- **No sincronizados** y sus iteradores son **fail-fast** (`ConcurrentModificationException`).

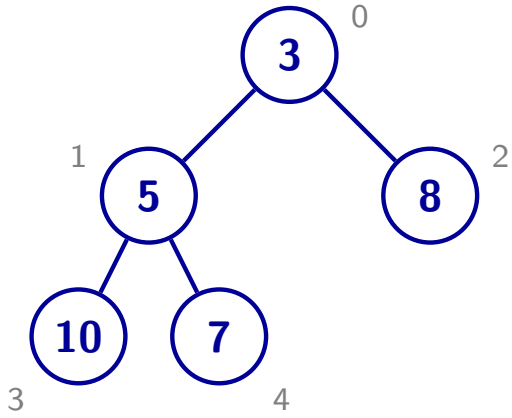
PriorityQueue: ¿Qué es?

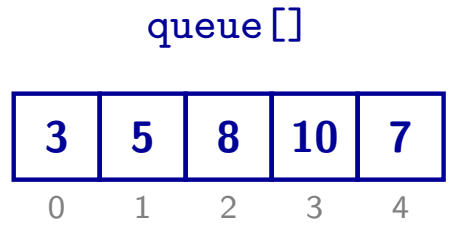
- Cola de prioridad implementada mediante un **Heap Binario de mínimos**.
- `poll()` siempre retorna el **menor** elemento según el orden (`Comparable` o `Comparator`).
- El heap es un **árbol binario completo** almacenado de forma eficiente en un **arreglo**.

Propiedades del heap:

- **Forma**: árbol completo (todos los niveles llenos salvo el último, que se llena de izquierda a derecha sin huecos).
- **Orden**: cada padre es $\leq$ que sus hijos $\Rightarrow$ el elemento mínimo está en la raíz.
- **No** es un BST: los hermanos no tienen ningún orden relativo entre sí.

Heap: Árbol vs Arreglo



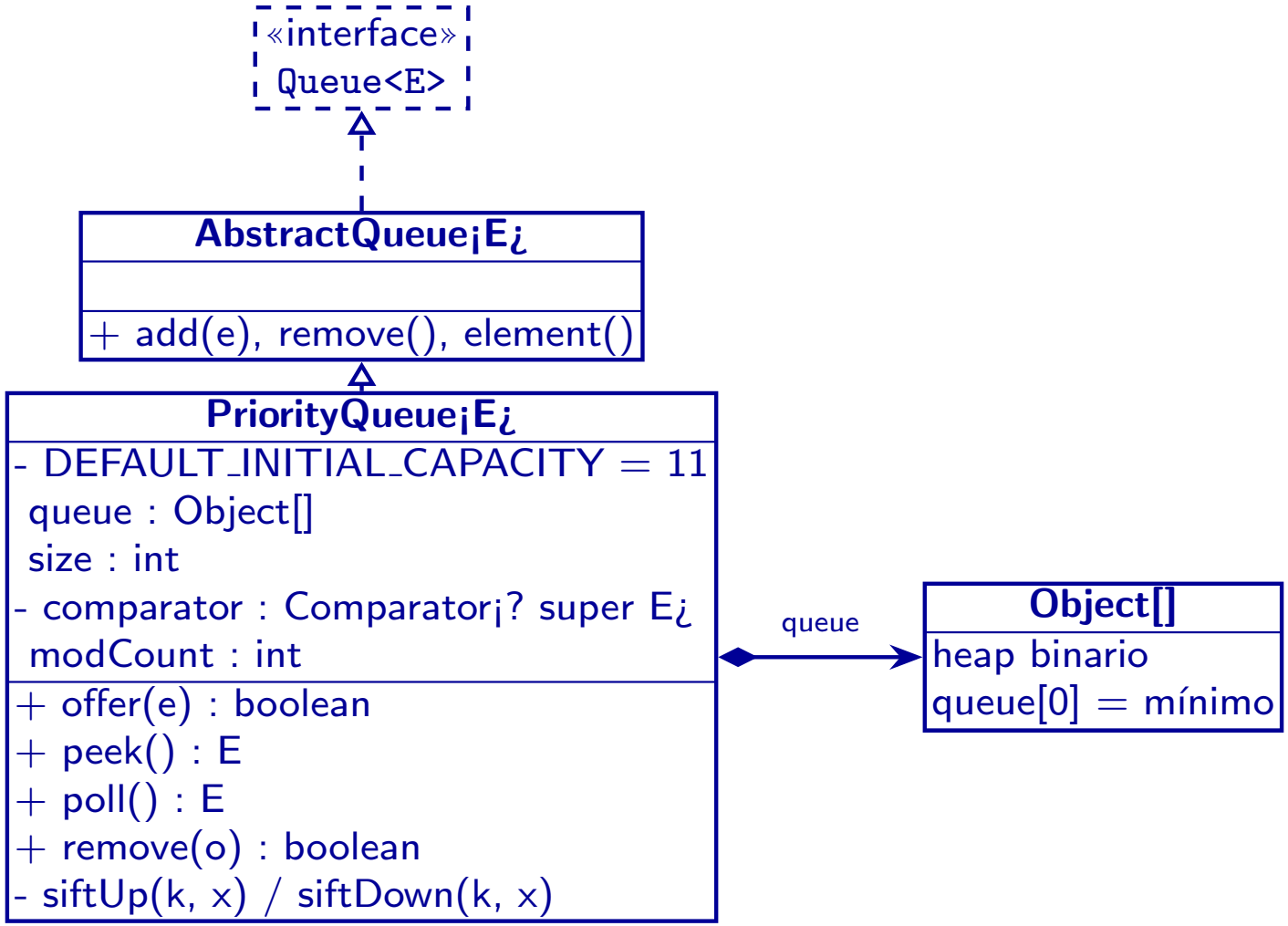


Para cualquier nodo en el índice i:

$$\text{padre}(i) = \left\lfloor \frac{i-1}{2} \right\rfloor \qquad \text{izq}(i) = 2i+1 \qquad \text{der}(i) = 2i+2$$

- No se requieren punteros: las relaciones están **implícitas** en los índices.
- Los nodos con al menos un hijo son los índices $0 \ldots \lfloor n/2 \rfloor - 1$.

Estructura interna (UML)



Estructura interna: PriorityQueue (Java)

```
public class PriorityQueue<E> extends AbstractQueue<E>
        implements java.io.Serializable {

    private static final int DEFAULT_INITIAL_CAPACITY = 11;

    // Heap binario: los hijos de queue[n] son
    // queue[2*n+1] y queue[2*n+2]. queue[0] es el minimo.
    transient Object[] queue;

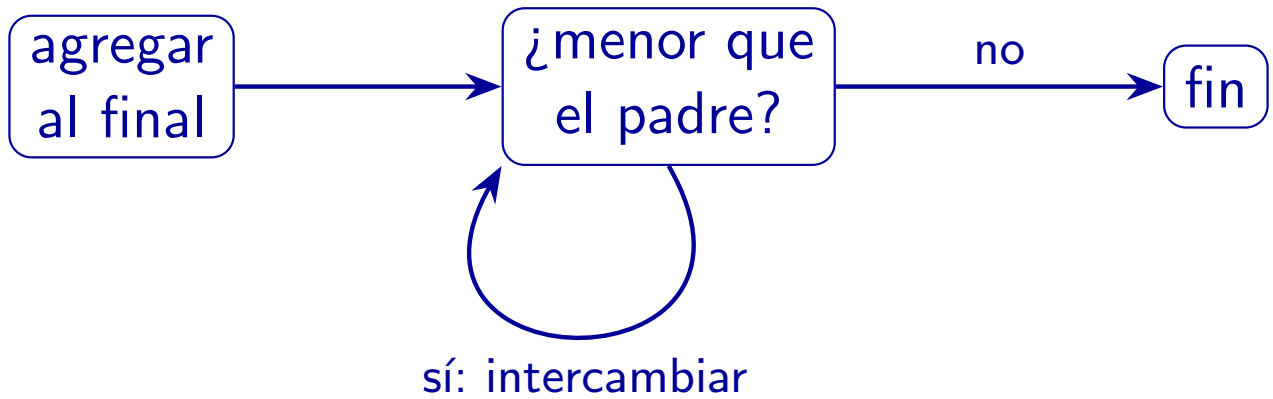
    int size;                                   // cantidad
        de elementos
    private final Comparator<? super E> comparator; // null
        = natural
    transient int modCount;                     // fail-fast

    public PriorityQueue() {
        this(DEFAULT_INITIAL_CAPACITY, null);
    }
    public PriorityQueue(Comparator<? super E> comparator) {
        this(DEFAULT_INITIAL_CAPACITY, comparator);
    }
}
```

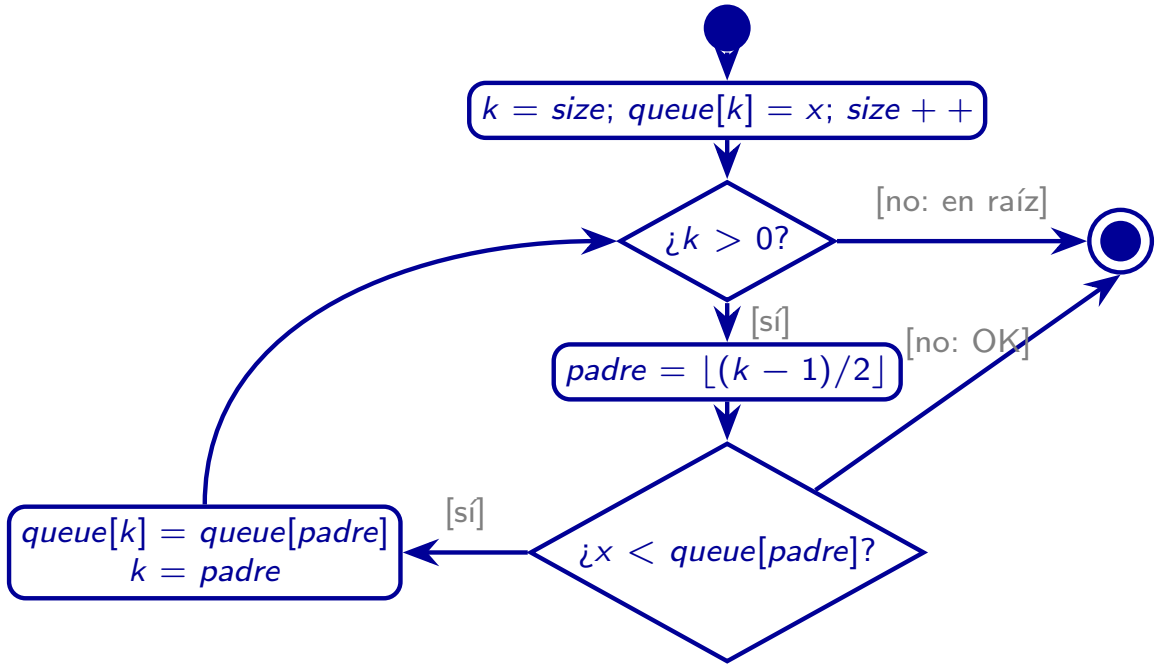
Inserción (`offer`): Definición

1. Si el arreglo está lleno, se redimensiona automáticamente (`grow`).
2. El elemento se coloca en la **primera posición libre** (índice `size`) $\Rightarrow$ se preserva la **forma** de árbol completo.
3. **Rebalanceo** – `siftUp`: mientras sea **menor que su padre**, se intercambia con él (asciende).

Costo: asciende a lo más la altura del árbol $\Rightarrow O(\log n)$.



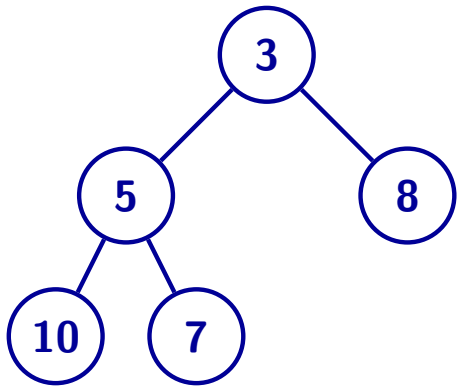
Inserción (offer): Diagrama de Actividad UML (siftUp)

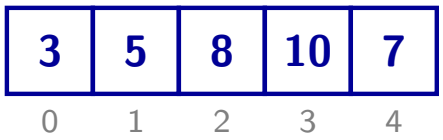


El nuevo elemento asciende por el árbol hasta restablecer la propiedad de heap ($O(\log n)$).

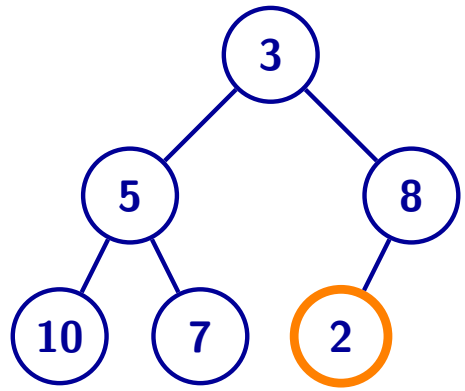
Inserción: Ejemplo paso a paso (1/2)

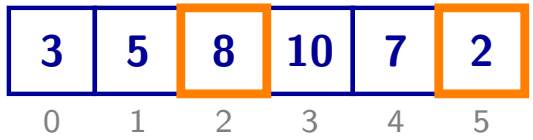
`offer(2)` sobre el heap `[3, 5, 8, 10, 7]`





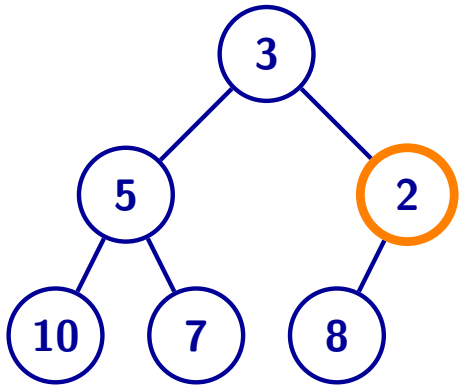
Heap inicial

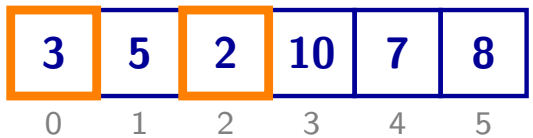




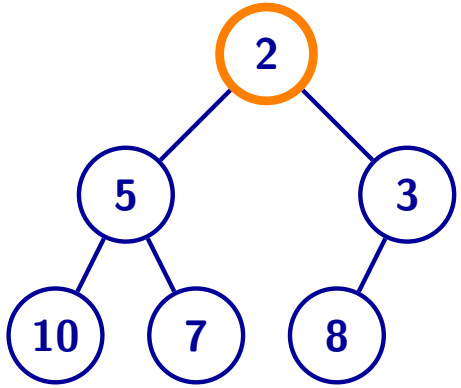
Paso 1: 2 va al índice 5
padre $= (5-1)/2 = 2$ (valor 8)
$2 < 8 \Rightarrow$ intercambiar

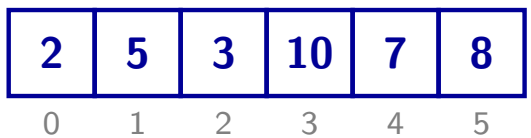
Inserción: Ejemplo paso a paso (2/2)





Paso 2: 2 está en el índice 2
padre $= (2-1)/2 = 0$ (valor 3)
$2 < 3 \Rightarrow$ intercambiar





Paso 3: 2 llegó a la raíz
(índice 0) $\Rightarrow$ fin
¡Heap válido!

Inserción: Java (offer)

```
public boolean offer(E e) {
    if (e == null)
        throw new NullPointerException();
    modCount++;
    int i = size;
    if (i >= queue.length)
        grow(i + 1);                    // +2 si es chico, +50%
            si es grande
    siftUp(i, e);                       // REBALANCEO hacia arriba
    size = i + 1;
    return true;
}
```

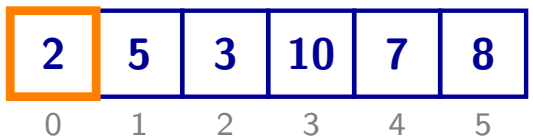
Rebalanceo: Java (siftUpComparable)

```
private void siftUpComparable(int k, E x) {
    Comparable<? super E> key = (Comparable<? super E>) x;
    while (k > 0) {
        int parent = (k - 1) >>> 1;          // indice del
            padre
        Object e = queue[parent];
        if (key.compareTo((E) e) >= 0)       // ya no es
            menor: parar
            break;
        queue[k] = e;                        // el padre baja
        k = parent;
    }
    queue[k] = key;                          // posicion final
        encontrada
}
```

Búsqueda: Definición y complejidad

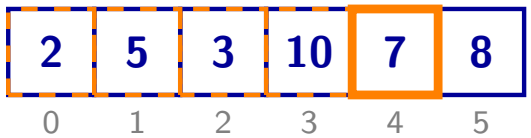
- `peek()`: el elemento mínimo siempre reside en la raíz `queue[0]` $\Rightarrow$ acceso inmediato en $O(1)$.
- `contains(o)`: un heap **no** es un árbol de búsqueda binaria. No existe orden entre hermanos $\Rightarrow$ requiere recorrido **lineal** del arreglo, tiempo $O(n)$.

`peek()`



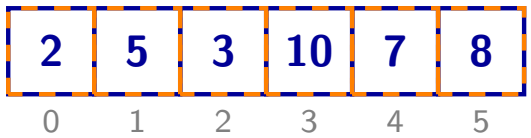
$\rightarrow$ **2** (1 acceso, $O(1)$)

`contains(7)`



$\rightarrow$ **true** (5 accesos)

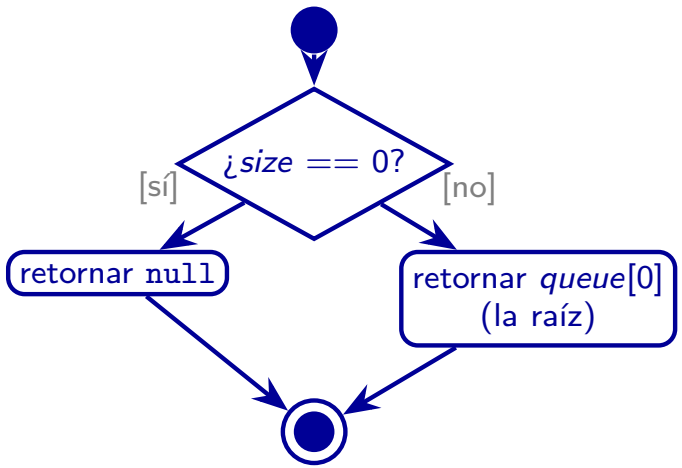
`contains(4)`



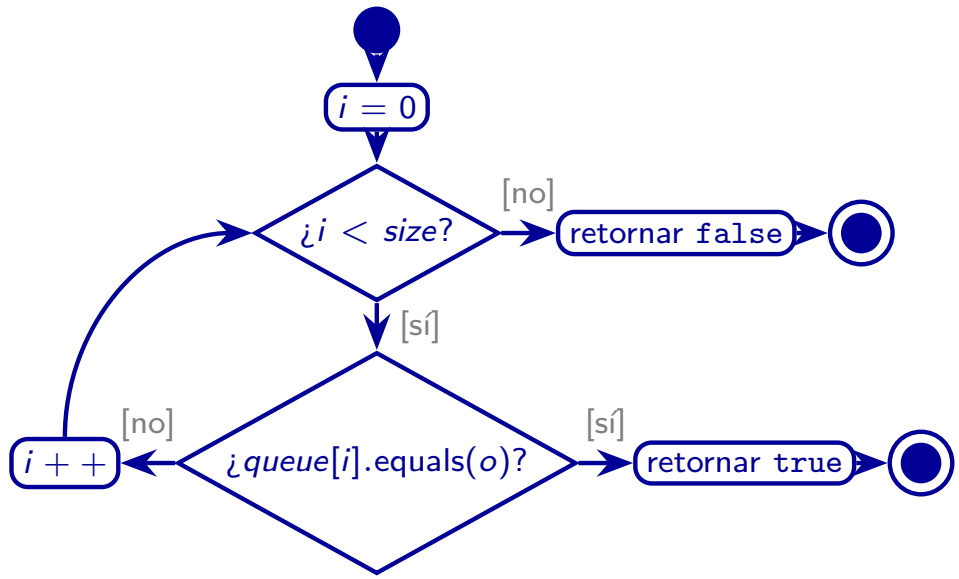
$\rightarrow$ **false** (6 accesos, $O(n)$)

Búsqueda: Diagrama de Actividad UML (peek / contains)

`peek()` – $O(1)$



`contains(o)` – $O(n)$



Búsqueda: Java (peek, contains)

```
public E peek() {
    return (E) queue[0];          // O(1): la raiz es el
        minimo
}

public boolean contains(Object o) {
    return indexOf(o) >= 0;
}

private int indexOf(Object o) {   // O(n): busqueda lineal
    if (o != null) {
        final Object[] es = queue;
        for (int i = 0, n = size; i < n; i++)
            if (o.equals(es[i]))
                return i;
    }
    return -1;
}
```

Eliminación (`poll`): Definición

`poll()` – extraer el mínimo:

1. Se guarda el valor de la raíz `queue[0]` (el resultado a retornar).
2. El **último** elemento se traslada a la raíz y se decrementa `size` $\Rightarrow$ se conserva la **forma** de árbol completo.
3. **Rebalanceo** – `siftDown`: mientras sea **mayor que el menor de sus hijos**, se intercambia con dicho hijo (desciende).

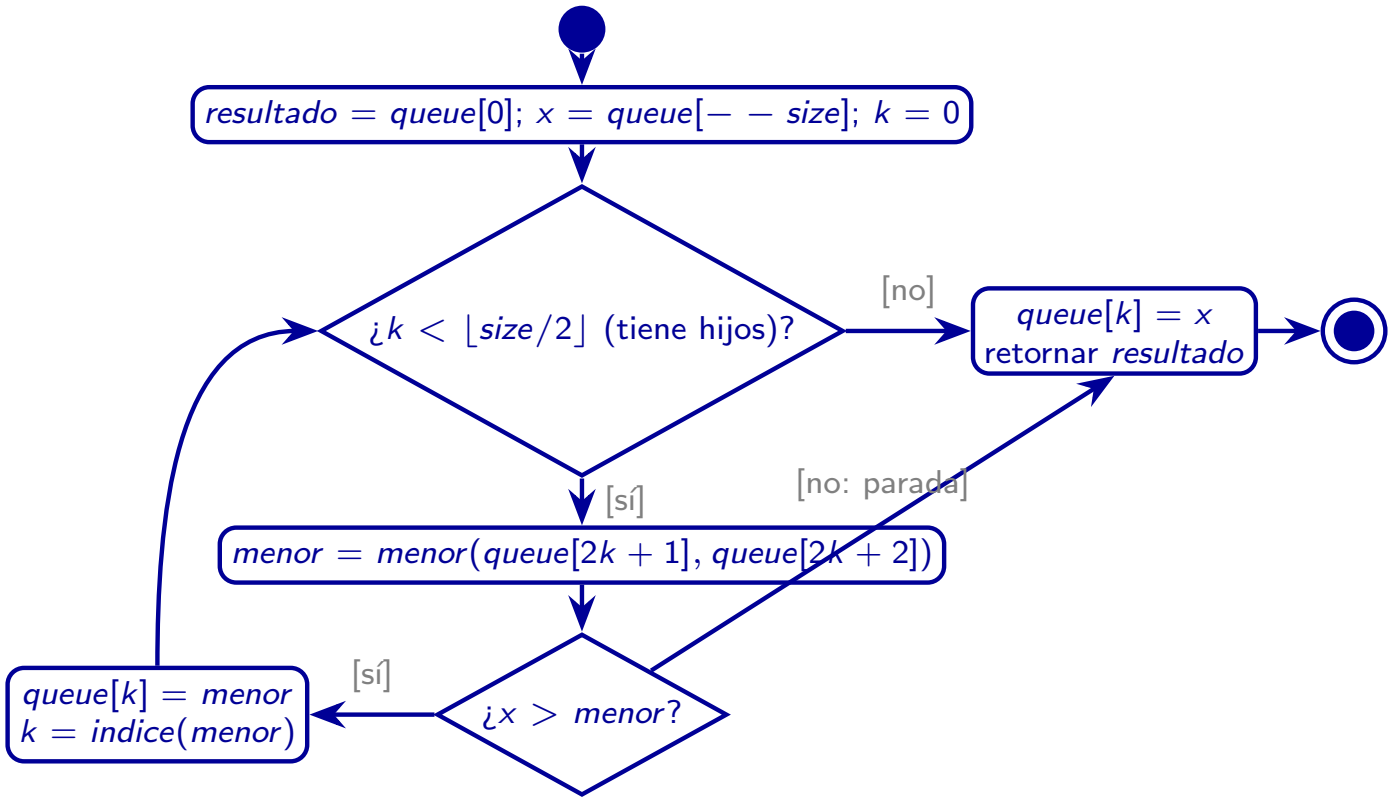
Costo: $O(\log n)$.

`remove(o)` – eliminar un elemento arbitrario:

1. Se localiza linealmente con `indexOf` ($O(n)$).
2. Se coloca el último en su posición y se llama a `siftDown`; si no bajó, se llama a `siftUp`.

Costo: $O(n)$ debido a la búsqueda.

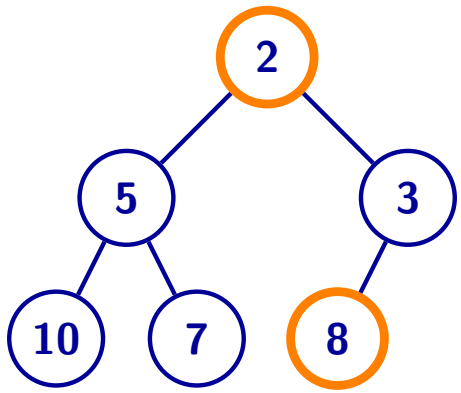
Eliminación (poll): Diagrama de Actividad UML (siftDown)

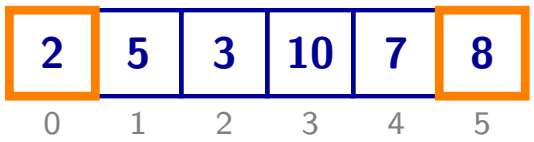


Desciende por el camino del menor hijo hasta restablecer la propiedad de heap ($O(\log n)$).

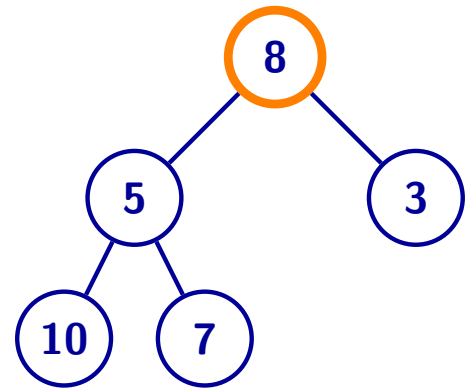
Eliminación: Ejemplo paso a paso (1/2)

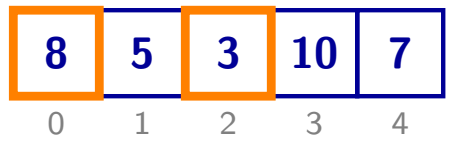
`poll()` sobre el heap `[2, 5, 3, 10, 7, 8]`





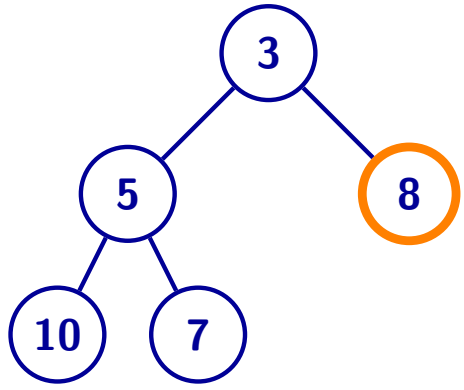
Paso 1: se retorna 2
el último (8) pasa a la raíz





Paso 2: hijos de 8: 5 y 3
menor hijo = 3 (índice 2)
$8 > 3 \Rightarrow$ intercambiar

Eliminación: Ejemplo paso a paso (2/2)



3	5	8	10	7
0	1	2	3	4

Paso 3: 8 en índice 2
$2 \geq \lfloor 5/2 \rfloor \Rightarrow$ no tiene hijos
¡Heap válido!

- El heap volvió al estado `[3, 5, 8, 10, 7]`.
- Al llamar a `poll()` sucesivamente, los valores salen: 3, 5, 7, 8, 10: **¡ordenados!** (Heapsort).

Eliminación: Java (poll)

```
public E poll() {
    final Object[] es = queue;
    final E result = (E) es[0];                    // el minimo
        esta en la raiz
    if (result != null) {
        modCount++;
        final int n = --size;
        final E x = (E) es[n];                     // ultimo
            elemento
        es[n] = null;
        if (n > 0)
            siftDown(0, x);                        // REBALANCEO
                hacia abajo
    }
    return result;
}
```

Rebalanceo: Java (siftDownComparable)

```
private void siftDownComparable(int k, E x) {
    Comparable<? super E> key = (Comparable<? super E>) x;
    int half = size >>> 1;                    // nodos con
        hijos
    while (k < half) {
        int child = (k << 1) + 1;             // hijo
            izquierdo
        Object c = queue[child];
        int right = child + 1;
        if (right < size &&
            ((Comparable<? super E>) c).compareTo((E)
                queue[right]) > 0)
            c = queue[child = right];         // elegir el
                menor hijo
        if (key.compareTo((E) c) <= 0)
            break;                            // propiedad
                restablecida
        queue[k] = c;                         // el menor
            hijo sube
        k = child;
    }
    queue[k] = key;
}
```

Eliminación y rebalanceo: Java (`removeAt`, `heapify`)

```
E removeAt(int i) {                    // usado por remove(Object o)
    modCount++;
    int s = --size;
    if (s == i)                        // era el ultimo: solo se borra
        queue[i] = null;
    else {
        E moved = (E) queue[s];
        queue[s] = null;
        siftDown(i, moved);            // intentar bajar
        if (queue[i] == moved) {       // no bajo: quizas debe subir
            siftUp(i, moved);
            if (queue[i] != moved)
                return moved;
        }
    }
    return null;
}

// Construir un heap desde una coleccion en O(n)
private void heapify() {
    for (int i = (size >>> 1) - 1; i >= 0; i--)
        siftDown(i, (E) queue[i]);
}
```

Uso en Java: PriorityQueue

```
PriorityQueue<Integer> pq = new PriorityQueue<>();  // min-heap
pq.offer(3); pq.offer(5); pq.offer(8);
pq.offer(10); pq.offer(7); pq.offer(2);
System.out.println(pq);       // [2, 5, 3, 10, 7, 8] (arreglo!)
pq.peek();                    // 2
pq.poll();                    // 2 -> [3, 5, 8, 10, 7]

// Max-heap con comparador inverso
PriorityQueue<Integer> max =
        new PriorityQueue<>(Comparator.reverseOrder());

// Objetos propios con comparador
record Tarea(String nombre, int prioridad) {}
PriorityQueue<Tarea> tareas = new PriorityQueue<>(
        Comparator.comparingInt(Tarea::prioridad));
tareas.add(new Tarea("Estudiar EDA", 2));
tareas.add(new Tarea("Entregar tarea", 1));
tareas.poll();                // Tarea[Entregar tarea, 1]
```

PriorityQueue: Propiedades principales

- Es un **min-heap** por defecto: el primer elemento es el menor. Para un max-heap: `Comparator.reverseOrder()`.
- `offer`/`poll`: $O(\log n)$ `peek`: $O(1)$ `contains`/`remove(o)`: $O(n)$.
- **Admite elementos duplicados**.
- **No admite** `null` (`NullPointerException`).
- El **iterador** y `toString()` recorren el arreglo sin orden garantizado. Para extraerlos ordenados se debe usar `poll()` repetidamente.
- Capacidad inicial 11; crece dinámicamente ($+2$ si es pequeña, $+50\,\%$ si es grande).
- Construir desde una colección existente toma tiempo lineal $O(n)$ mediante `heapify`.
- **No sincronizada**: para entornos multihilo utilizar `PriorityBlockingQueue`.

Comparación

Característica	**TreeSet¡E¿**	**TreeMap¡K,V¿**	**PriorityQueue¡E¿**
Estructura interna	Rojo-Negro (vía TreeMap)	Árbol Rojo-Negro	Heap binario (arreglo)
Inserción	`add(e)` $O(\log n)$	`put(k,v)` $O(\log n)$	`offer(e)` $O(\log n)$
Búsqueda	`contains(o)` $O(\log n)$	`get(k)` $O(\log n)$	`contains(o)` $O(n)$
Elemento extremo	`first()` $O(\log n)$	`firstKey()` $O(\log n)$	`peek()` $O(1)$
Eliminación	`remove(o)` $O(\log n)$	`remove(k)` $O(\log n)$	`poll()` $O(\log n)$
Rebalanceo	Rotaciones + recoloreos	Rotaciones + recoloreos	`siftUp` / `siftDown`
Duplicados	No	Claves no, valores sí	Sí
Permite `null`	No	Claves no, valores sí	No
Iteración	En orden (inorden)	En orden por clave	Sin orden garantizado

- ¿Se requiere consultar rangos o mantener todos los datos ordenados? $\rightarrow$ `TreeSet` / `TreeMap`.
- ¿Solo se necesita extraer el elemento de mayor prioridad repetidamente? $\rightarrow$ `PriorityQueue`.

Códigos de ejemplo

Archivos Java incluidos en la carpeta:

- `EjemploTreeSet.java`: uso de `TreeSet` (navegación, comparadores personalizados).
- `EjemploTreeMap.java`: uso de `TreeMap` (frecuencias, búsqueda por tramos).
- `EjemploPriorityQueue.java`: min-heap, max-heap, tareas por prioridad y top-k.
- `ArbolRojoNegro.java`: implementación desde cero del Árbol Rojo-Negro con **traza** detallada de rebalanceo (reproduce los ejemplos de la cátedra).
- `MinHeap.java`: implementación desde cero del heap binario con traza de `siftUp`/`siftDown`.

